Satoshi's Wall

TECHNICAL WHITEPAPER

# The Permanent Bitcoin Message Board

Trustless, censorship-resistant inscription of human expression directly on Bitcoin Layer 1 — powered by OPNet smart contracts

**Abstract.** *Satoshi's Wall is a decentralised application that enables any Bitcoin wallet holder to inscribe a 64-character message permanently on Bitcoin Layer 1 via the OPNet protocol. The system enforces a single inscription per address, collects a 1,000-satoshi fee directly via Bitcoin's UTXO model, and provides a consumer-grade frontend with real-time wall updates. This document describes the protocol design, smart contract architecture, frontend implementation, and economic model.*

Version 1.0.0 · February 2026 OPNet Testnet · opt1sqqysd49aw7suh5epsvepc8qsy0698apg4s7p02zt

### Contents



## 1 Introduction

Bitcoin is the most secure, most permanent, and most decentralised data layer ever created. Its blockchain has never been successfully attacked, reversed, or censored since its genesis block in January 2009. Yet until now, there has been no way to harness Bitcoin's permanence for expressive, programmable applications without sacrificing one of its core properties.

Satoshi's Wall is a proof-of-concept and production-ready application demonstrating that Bitcoin Layer 1 can support consumer-facing smart contract applications with programmable rules, direct BTC monetisation, and a seamless user experience — all without leaving the Bitcoin security boundary.

The application allows any holder of a Bitcoin wallet (via the OPWallet extension) to inscribe exactly one 64-character message on the Bitcoin blockchain forever. The inscription is enforced by an on-chain smart contract that verifies the sender has not previously posted, the message is non-empty, the wall has not reached capacity, and a 1,000-satoshi payment has been included in the Bitcoin transaction.

# 2 The Problem: Impermanent Digital Expression

## 2.1 Platform Fragility

Human beings have always sought to leave permanent marks — from cave paintings to stone inscriptions. In the digital age, the tools for self-expression are powerful but fragile. Social media posts can be deleted in seconds. Blogs disappear when hosting fees lapse. Platforms shut down, are acquired, or censor content at will.

The average lifespan of a URL is approximately 75 days. Even the Internet Archive, which endeavours to preserve the web, is a centralised custodian subject to legal pressure and funding constraints.

## 2.2 Blockchain Inscriptions — Existing Limitations

Bitcoin Ordinals and BRC-20 tokens demonstrated enormous demand for on-chain inscription, generating billions in fees at peak. However, these protocols have significant limitations:

- **No programmability** — Ordinals are static; they cannot enforce rules like "one inscription per address"
- **No direct monetisation** — No trustless mechanism for collecting fees without a separate token or bridge
- **High variance fees** — During periods of high demand, Ordinals inscription costs spiked above $50 per inscription
- **No application logic** — Cannot implement features like leaderboards, limits, or fee-gating at the protocol level

## 2.3 The Opportunity

OPNet introduces Turing-complete smart contracts executed within Bitcoin's consensus via Tapscript-encoded calldata. This enables a new class of applications that were previously impossible on Bitcoin: programmable, fee-collecting, state-maintaining contracts that operate entirely within the Bitcoin security boundary.

# 3 Background: OPNet on Bitcoin Layer 1

### 3.1 OPNet Architecture Overview

OPNet is a Bitcoin Layer 1 smart contract protocol that executes WebAssembly (WASM) bytecode within Bitcoin transactions. Contracts are written in AssemblyScript (a TypeScript-like language) and compiled to WASM. The compiled WASM is deployed via a two-phase Bitcoin transaction:

```
Phase 1 (Commit): Funding TX → locks sats in a P2TR address derived from contract bytecode Phase 2 (Reveal): Reveal TX → spends the P2TR UTXO, encodes WASM in Tapscript witness data Contract interaction (read): btc_call RPC → OPNet node executes contract method → returns encoded result Contract interaction (write): Phase 1: Commit TX (fund the interaction address) Phase 2: Reveal TX (Tapscript contains calldata + method selector)
```

### 3.2 Key Differences from EVM

| Property | EVM (Ethereum) | OPNet (Bitcoin) |
|---|---|---|
| Language | Solidity / Vyper | AssemblyScript (compiled to WASM) |
| Execution | EVM bytecode, gas metered | WASM, Tapscript-encoded |
| Value transfer | `msg.value`, payable functions | UTXO outputs in `Blockchain.tx.outputs` |
| Contract balance | Native (address.balance) | Not supported — verify-don't-custody model |
| Signatures | ECDSA (secp256k1) | ML-DSA (post-quantum) + Bitcoin ECDSA |
| Storage | EVM storage slots (256-bit) | Pointer-based StoredU256 / StoredMapU256 |
| Security | Ethereum consensus | Bitcoin PoW (most secure chain in existence) |

### 3.3 The UTXO Payment Model

OPNet contracts cannot hold or send BTC. Instead, they use a "verify-don't-custody" pattern. When a user calls a payable-style method, the Bitcoin transaction includes a UTXO output paying the required amount to the target address. The contract verifies this output exists before updating state:

```
// In AssemblyScript contract
private requireMinPayment(): u64 {
    const outputs: TransactionOutput[] = Blockchain.tx.outputs;
    for (let i = 0; i < outputs.length; i++) {
        const out = outputs[i];
        if (out.hasTo && out.to === TREASURY && out.value >= 1000) {
            return out.value; // payment verified
        }
    }
    throw new Revert('Insufficient payment');
}
```

## 4 Protocol Design

### 4.1 Core Invariants

The Satoshi's Wall protocol enforces three immutable invariants at the contract level:

**INVARIANT 1 — UNIQUENESS**

Each Bitcoin address may inscribe exactly one message. The `_hasPosted` mapping stores `index + 1` for each posted address; zero means not posted. This is checked before any state changes.

**INVARIANT 2 — PAYMENT**

Every `postMessage` call must include a UTXO output of at least 1,000 satoshis to the treasury address within the same Bitcoin transaction. The contract verifies this by iterating `Blockchain.tx.outputs` before executing any write.

**INVARIANT 3 — CAPACITY**

The wall holds a maximum of 10,000 messages. Once full, all `postMessage` calls revert. This creates artificial scarcity and a permanent historical record of the first 10,000 inscribers.

### 4.2 Message Encoding

Messages are stored as two 256-bit (32-byte) unsigned integers — a natural fit for OPNet's storage model. The encoding is straightforward UTF-8 to big-endian bytes:

```
// Encode: string → [chunk1, chunk2]
function encodeMessage(text: string): [bigint, bigint] {
    const bytes = new TextEncoder().encode(text.slice(0, 64));
    const padded = new Uint8Array(64);
    padded.set(bytes); // zero-padded to 64 bytes
    return [
        bytesToBigInt(padded.slice(0, 32)), // chunk1
        bytesToBigInt(padded.slice(32, 64)), // chunk2
    ];
}

// Decode: [chunk1, chunk2] → string
function decodeMessage(chunk1: bigint, chunk2: bigint): string {
    const bytes = new Uint8Array(64);
    bytes.set(bigIntToBytes(chunk1, 32), 0);
    bytes.set(bigIntToBytes(chunk2, 32), 32);
    let end = bytes.length;
    while (end > 0 && bytes[end-1] === 0) end--;
    return new TextDecoder().decode(bytes.slice(0, end));
}
```

This encoding is collision-free for all 64-character UTF-8 strings (padded with null bytes). The null-trim on decode cleanly recovers the original string.

## 5 Smart Contract Architecture

### 5.1 Storage Layout

```
// Pointer allocation (each Blockchain.nextPointer call increments a counter)
messageCountPointer (0) → StoredU256      — total messages posted
sendersPointer      (1) → StoredMapU256   — index → sender address (as u256)
blocksPointer       (2) → StoredMapU256   — index → Bitcoin block number
texts1Pointer       (3) → StoredMapU256   — index → message chunk1
texts2Pointer       (4) → StoredMapU256   — index → message chunk2
hasPostedPointer    (5) → AddressMemoryMap — address → (index+1) or 0
totalRaisedPointer  (6) → StoredU256      — cumulative sats paid to treasury
```

### 5.2 Method Specification

#### POSTMESSAGE(CHUNK1, CHUNK2) → BOOL

Selector: `635734b9`

1. Call `requireMinPayment()` — iterates outputs, reverts if no 1,000 sat output to treasury
2. Check `_hasPosted[sender] == 0` — reverts if duplicate
3. Check `messageCount < 10,000` — reverts if full
4. Check `chunk1 != 0 || chunk2 != 0` — reverts if empty
5. Write: `_senders[count]`, `_blocks[count]`, `_texts1[count]`, `_texts2[count]`
6. Write: `_hasPosted[sender] = count + 1`
7. Write: `_messageCount += 1`, `_totalRaised += satsPaid`

#### GETMESSAGES(OFFSET, LIMIT) → BYTES

Selector: `77983818` — Returns a batch of up to 50 messages. Response: `uint32 count` (4 bytes) + N entries of 128 bytes each. Each entry: `sender(32) + blockNumber(32) + chunk1(32) + chunk2(32)`.

#### GETTOTALRAISED() → UINT256

Selector: `99e14bc8` — Returns `_totalRaised`, the actual cumulative satoshis verified by the contract across all posts.

## 6 Frontend Architecture

### 6.1 Stack

| Layer | Technology | Purpose |
|---|---|---|
| Framework | React 18 + TypeScript | Component rendering + type safety |
| Build | Vite 5 | Dev server, bundling, HMR |
| Styling | Tailwind CSS v4 + custom CSS | Utility classes + glass morphism |
| Wallet | OPWallet ( `window.opnet` ) | Bitcoin signing, UTXO management |
| Contract reads | Native `fetch()` + `btc_call` | Browser-safe, bypasses opnet undici |
| Contract writes | opnet npm + `getContract` | Simulation + transaction building |

### 6.2 Critical Browser Compatibility Fix

The `opnet` library's `JSONRpcProvider` uses Node.js's `undici` HTTP client internally. This client does not work in browser environments and is also intercepted by OPWallet, which injects itself as the provider for all opnet calls.

The solution: all read calls ( `getMessages` , `getMessageCount` , `getTotalRaised` ) use native `fetch()` directly against the `btc_call` JSON-RPC endpoint, bypassing the opnet library entirely for reads. The opnet library is used exclusively for writes (simulation + `sendTransaction` ).

### 6.3 Polling Strategy

```
Every 5 seconds: ↓ fetch getMessageCount() ← 1 lightweight call (32 bytes) ↓ if count changed: ↓ fetch getMessages(offset, 50) ← 1 batch call ↓ fetch getTotalRaised() ← parallel with getMessages ↓ setMessages() + setStats() ↓ if count unchanged: ↓ update totalRaised only (silent background refresh)
```

This approach reduces RPC calls from *N+1 per refresh* (one count call + N individual getMessage calls) to *2 calls maximum* (one count check + one batch fetch), regardless of how many messages are on the wall.

### 6.4 OPWallet Patch

OPWallet's `signInteraction` method checks for the presence of `signer` and `mldsaSigner` keys in the transaction parameters object, even when they are `null` . This causes it to throw "signer is not

allowed in interaction parameters" for frontend transactions (which must use `null` signers — the wallet handles signing).

Fix: patch `node_modules/opnet/browser/index.js` to use spread syntax that omits the keys entirely when null:

```
// Before (fails):
{ signer: e.signer, mldsaSigner: e.mldsaSigner }

// After (works):
{
  ...(e.signer != null ? { signer: e.signer } : {}),
  ...(e.mldsaSigner != null ? { mldsaSigner: e.mldsaSigner } : {}),
}
```

## 7 Payment Model

### 7.1 The Verify-Don't-Custody Pattern

OPNet contracts cannot hold, receive, or send BTC. This is a fundamental design difference from EVM-based chains. Satoshi's Wall uses the following payment architecture:

```
User submits postMessage: Bitcoin Transaction: ├── Input: User's UTXO (funding the transaction) ├── Output 0: ~1,000 sats → Treasury address ← contract verifies this ├── Output 1: Calldata (Tapscript with postMessage + message bytes) └── Output 2: Change → User's address Contract execution: 1. Iterates Blockchain.tx.outputs 2. Finds output to treasury with value >= 1,000 sats 3. Executes state writes 4. Treasury wallet receives BTC at the Bitcoin UTXO level No bridge. No wrap. No escrow. No custody risk.
```

### 7.2 Frontend Payment Integration

The frontend must instruct both the simulator and OPWallet about the extra treasury output:

```
// Step 1: Tell simulator about the output (so contract can verify during sim)
contract.setTransactionDetails({
    inputs: [],
    outputs: [{
        value: 1000n,   // BigInt (satoshis)
        index: 1,
        flags: 1,       // 0b00000001 = TransactionOutputFlags.hasTo
        to: TREASURY_ADDRESS,
    }],
});

// Step 2: Simulate (contract verifies the output, checks hasPosted, etc.)
const sim = await contract.postMessage(chunk1, chunk2);

// Step 3: Send with the real extra output included in the Bitcoin transaction
await sim.sendTransaction({
    signer: null,           // OPWallet handles signing on frontend
    mldsaSigner: null,
    refundTo: wallet.address,
    maximumAllowedSatToSpend: 20000n,
    network: ACTIVE_NETWORK,
    extraOutputs: [{ address: TREASURY_ADDRESS, value: 1000 }],  // number, not bigint
});
```

## 8 Security Considerations

### 8.1 Contract-Level Protections

| Attack Vector | Mitigation |
|---|---|
| Duplicate posting from same address | `_hasPosted[sender] != 0` check with revert before any writes (checks-effects-interactions pattern) |
| Empty message spam | Reverts if `chunk1 == 0 && chunk2 == 0` |
| Wall overflow beyond 10,000 | Hard capacity check against `MAX_MESSAGES = 10,000` |
| Fee bypass | Payment check is the FIRST operation in `postMessage`; all other checks follow |
| Treasury address spoofing | Treasury address is a compile-time constant — cannot be changed post-deploy |
| Integer overflow in totalRaised | Uses `SafeMath.add` which reverts on overflow |

### 8.2 Frontend Security

- All message text rendered via React's virtual DOM — no `dangerouslySetInnerHTML`, XSS-safe
- Message length enforced both client-side (textarea `maxLength`) and on-chain (64-byte encoding silently truncates)
- No private keys or mnemonics stored in application state
- RPC calls use `AbortSignal.timeout(15000)` — no hanging connections

### 8.3 Known Limitations

- **Treasury address immutability:** The treasury address is hardcoded at deployment. A compromised deployer wallet cannot be rotated without redeploying the contract and losing historical data.
- **No message deletion:** By design, messages cannot be removed. This is a feature — but offensive content once inscribed cannot be retracted.
- **opnet library patch:** The OPWallet compatibility patch modifies `node_modules` files which are not tracked in version control. A post-install script is required for reproducible builds.

## 9 Economic Analysis

### 9.1 Cost Structure Per Post

| Cost Component | Amount | Recipient |
|---|---|---|
| Treasury fee (fixed) | 1,000 sats | Satoshi's Wall treasury |
| Bitcoin network fee (variable) | ~5,000–15,000 sats | Bitcoin miners |
| OPNet commit TX fee | Included in network fee | Bitcoin miners |
| **Total per post** | **~6,000–16,000 sats** | — |

### 9.2 Scarcity and Value

The hard cap of 10,000 messages creates permanent scarcity. Each of the 10,000 wall slots is unique by block position and sender address. The earliest inscribers — visible in the Hall of Fame leaderboard — hold the most historically significant positions.

At full capacity (10,000 messages), the treasury will have received exactly **10,000,000 satoshis (0.1 BTC)** — approximately $10,000 at $100,000/BTC, or $100,000 at $1,000,000/BTC.

### 9.3 Revenue Model

> **PROJECTION AT $100,000/BTC**
>
> Each message = 1,000 sats = ~$1.00 in treasury revenue.
> Full wall (10,000 messages) = 0.1 BTC = ~$10,000 to treasury.
> At $1M/BTC: full wall = $100,000 to treasury.

## 10 Roadmap & Future Work

### Phase 1 — Testnet (Complete)

- Full AssemblyScript smart contract with payment enforcement
- Batch message fetching, real total-raised tracking
- Consumer-grade React frontend with glass morphism UI
- OPWallet integration with browser-compatible RPC
- Leaderboard, live feed, optimistic updates

### Phase 2 — Mainnet Launch

- Independent security audit of smart contract
- Mainnet deployment (Bitcoin mainnet via OPNet)
- Certificate of Inscription: OP721 NFT minted per message
- Social sharing with verifiable on-chain proof links

### Phase 3 — Ecosystem

- Public API for third-party integrations
- Embeddable wall widget
- Multiple themed walls (Bitcoin history, dedications, art)
- Treasury governance: community vote on fee allocation
- Mobile PWA with OPWallet deep-link support

## 11 Conclusion

Satoshi's Wall demonstrates that Bitcoin Layer 1 is no longer limited to simple value transfer. OPNet's programmable smart contract layer — built within Bitcoin's existing consensus without a sidechain, L2, or bridge — enables a new class of applications that are simultaneously as secure as Bitcoin and as expressive as Ethereum.

By combining a carefully designed AssemblyScript contract with a consumer-grade frontend, we have produced a product that is:

- **Truly permanent** — stored in Bitcoin's immutable UTXO set
- **Truly trustless** — rules enforced by contract, not by a server
- **Truly Bitcoin-native** — fees paid and received in BTC, no token required
- **Consumer-ready** — accessible to anyone with OPWallet and testnet BTC

The inscription of the first messages on Satoshi's Wall is a small but meaningful milestone: the first time ordinary Bitcoin users could interact with a programmable, fee-collecting Bitcoin smart contract through a polished, consumer-friendly interface. We believe this is a preview of what Bitcoin will look like in the next decade.

> **A NOTE ON PERMANENCE**
>
> Every message posted to Satoshi's Wall will persist as long as a single node runs Bitcoin. That could be centuries. The question is not whether your message will survive — it's whether you'll

> take 60 seconds to leave one.

---

---